Security in Software Applications

Language-Based Security

SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

https://danielefriolo.github.io

Language-based Security

Programming Languages can provide security features and guarantees

- safety guarantees
  - memory safety
  - type safety (different levels depending on expressivity)
  - thread safety
- Access Control (in some form)
  - visibility and access restrictions (public, private, …)
  - sandboxing mechanisms
- Information Flow Control (in some form)
  - JFlow/Jif or next generation Javascript

These features often interdependent (type needs memory, sandboxing needs memory and type, …)

Other ways to help

A programming language can help security also if it

- offers good APIs and libraries
  - API with parametrized queries for SQL
  - secure string libraries for C
- supports external languages
- offers useful features
  - such as exceptions and their handling
- makes assurance easier
  - understand code in modular way …

Insecure programs can be written in ANY language (forget input validation, flaw in program logic, …)

Safety vs. Security

sometimes confused

- SAFETY: system protected against accidental failures
- SECURITY: system protected against induced failures (active attacker)

separation line not clear, but good for safety also good for security

General Idea

when does the assignment

```
a[i] = (byte)b ;
```

make sense?

General Idea

when does the assignment

```
a[i] = (byte)b ;
```

make sense?

- `a` must be a non-null byte array
- `i` a non-negative integer < arraylength
- `b` should be a byte or converted to one

General Idea

when does the assignment

```
a[i] = (byte)b ;
```

make sense?

`a` must be a non-null byte array

`i` a non-negative integer < arraylength

`b` should be a byte or converted to one

Two approaches

- the programmer is responsible for verifying these conditions
  - unsafe approach
- the language is responsible for checking these conditions
  - safe approach

SAFE programming languages

Safe programming languages

- impose some discipline (via restrictions) on the programmer
- offer **abstractions** (enforced with associated guarantees) to the programmer

reduced freedom and flexibility in exchange for added safety and clearer understanding

Example abstractions

- programming languages constructs provide *abstraction from CPU instructions*
- programming language variables and types provide *abstraction of memory*
- procedure call mechanism provides abstraction over *call stack mechanism*
- …

Example OS abstractions

- OS provides virtual memory as abstraction of raw memory
- OS provides process as abstraction of CPU and memory
- OS provide security by
  - separating processes
  - controlling access to resources by process
  - BUT this security breaks whenever these abstractions break !

Abstractions

- abstractions are crucial to reduce complexity
  - reducing complexity reduces **bugs**

- abstractions provide security (*access control*)
  - memory access control by paging/segmentation
  - access to files (bits of hard disk) by processes

provided the abstractions are rigorously enforced

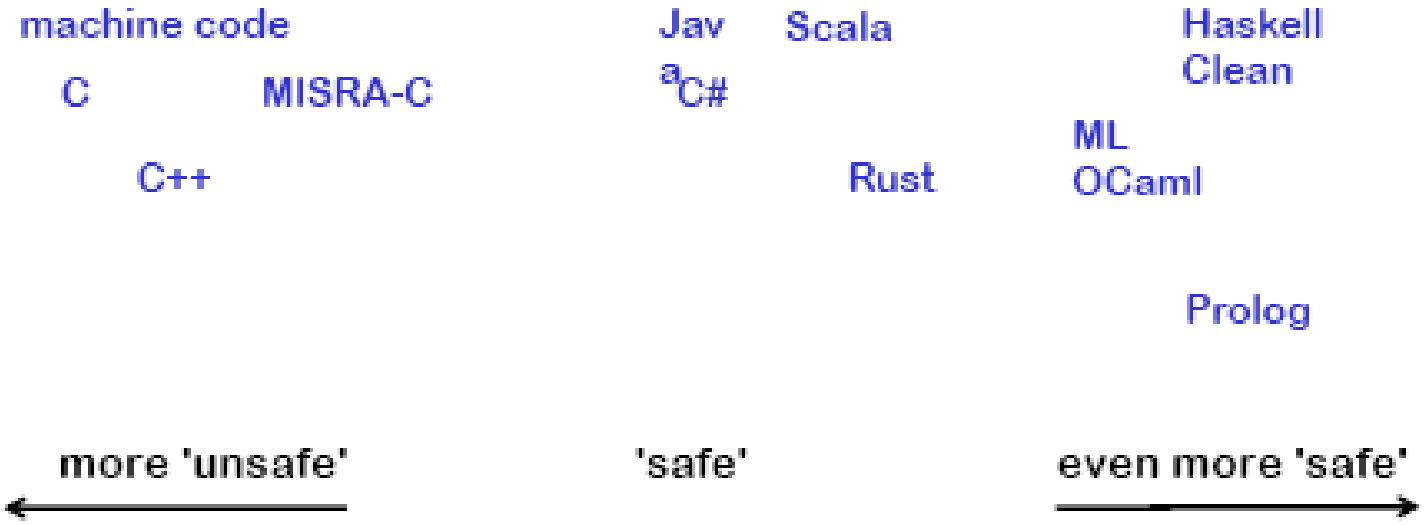
- but … some abstractions may be broken
  - stack overflows to break the procedure call mechanism
  - uninitialized virtual memory may leak information
  - timing attacks may reveal the virtual memory abstraction

General definition of Safety (?)

a programming language can be considered safe if

1. the abstractions provided by the language can be trusted
   - enforced and cannot be broken
   - boolean always `true` or `false`, never `35` or `null`, independently of representation `0x00 / true`
2. programs have precise and well-defined semantics
   - undefined behavior is source of trouble
3. the behavior of programs can be understood in a modular way

unsafe → safe languages



- oversimplistic
- easier if data is immutable (no side-effects) as in functional languages

dimensions and levels of safety

memory safety, type safety, thread safety, arithmetic safety, non-null safety, aliasing safety, ...

within each dimension, there can be (increasing) levels of safety

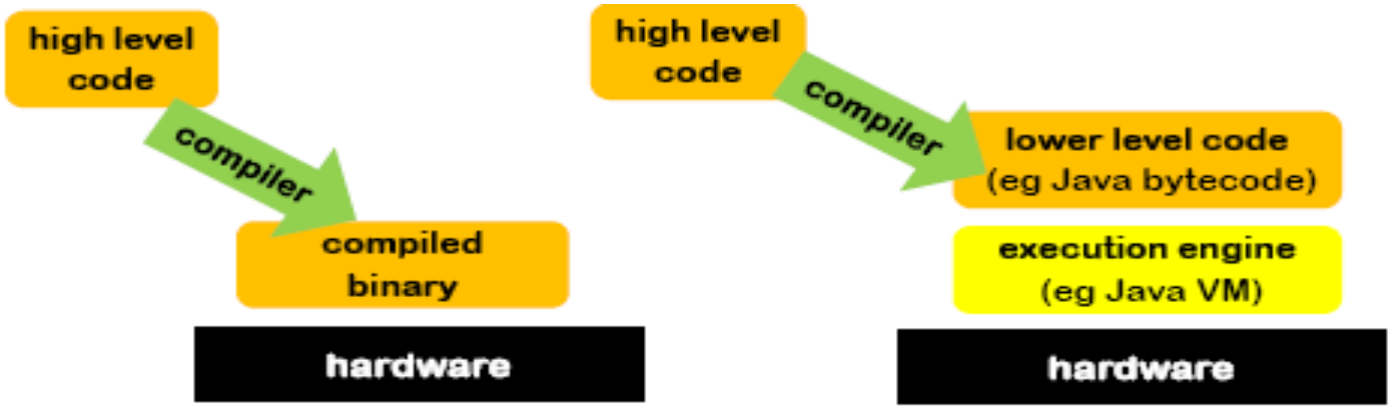
- e.g., index of array out-of-bounds
  1. let attacker inject arbitrary code
  2. possibly crash program or corrupt data
  3. definitely crash program
  4. throw exception, caught by the program for graceful handling
  5. ruled-out at compile-time

Safety : how?

Mechanisms to provide safety include

- compile time checks (e.g., type checking)
- run time checks (for array bounds, null pointers, runtime type checks, ...)
- garbage collector for automated memory management (no need to `free()` dynamic memory)
- execution engine
  - in JVM, bytecode verifier to type-check code, runtime checks, garbage collector invoked periodically

Safety : how?



Compiled binary runs on bare hardware
Any defensive measures must be compiled into the code

Execution engine isolates code from hardware
The programming language is still "there" at runtime and the execution engine can provide checks at runtime

Memory Safety

A programming language is *memory-safe* if it guarantees that

1. programs can never access *unallocated* or *de-allocated* memory
   - hence, no segmentation faults at runtime
   - so: OS access control for memory not necessary (if there are no bugs in execution engine)
2. (maybe) programs can never access uninitialized memory
   - so there is no need to zero-out memory before deallocating to avoid information leaks (if there …)

Memory Safety

Unsafe language features that can break memory-safety

1. unchecked array bounds
2. pointer arithmetic
3. null pointers (only when behavior is undefined)

Pointers in C

Folklore:

- dereferencing a NULL pointer will crash the program

but the C standard says

- the result of dereferencing a null pointer is undefined

so the program *may* crash but anything else may happen

- (see CERT Secure Coding Guidelnes for C for discussion on security vulnerability in PNG library caused a null pointer dereference that did not crash)

From C standard: ISO/IEC 9899:2011

*"If an invalid value has been assigned to the pointer, the behavior of the unary operator * is undefined.[102]*

*[102] Among the invalid values for dereferencing a pointer by the unary * operator are a null pointer, [...]"*

more from the standard

"*A null-pointer constant is either an integral constant expression that evaluates to zero (such as* `0` *or* `0L`*) or a value of type* `nullptr_t` *(such as* `nullptr`*)."*

Memory Safety

Unsafe language features that can break memory-safety

1. unchecked array bounds
2. pointer arithmetic
3. null pointers (only when behavior is undefined)
4. manual memory management
   - de-allocation e.g., `free()` in C causing dangling pointers, use-after-free and double-free bugs
   - can be avoided by
     - not using the heap at all (in MISRA C)
     - automating it with garbage collection
       - first used in LISP in 1959, then accepted with Java in 1995

Types and type safety

TYPES assert invariant properties of elements of a program

- this variable will always hold an integer
- this function will always return an object of class C
- this array will never store more than 10 elements

Type checking verifies these assertions and can be done

- at compile time (static typing)
- at runtime (dynamic typing)

Type soundness (type-safety / strong typing) refers to a system that guarantees that the assertions hold at runtime

Type safety

Type-safe programming languages guarantee that programs that pass the type-checker verification can only manipulate data in ways allowed by their type

- *booleans* are not multiplied
- *integers* are not dereferenced
- references not the argument of square roots
- in OO languages, never `Method_not_found` errors at runtime

Memory and type safety

Programming languages can be

- memory-safe, typed and type safe
  - Java, C#, Rust, Go, and functional languages such as Haskell, ML, Clean, F#
- memory-safe and untyped
  - LISP, Prolog, many interpreted languages
- memory-unsafe, typed and type unsafe
  - C, C++
    - pointer arithmetic can break any guarantee by the type system
    - impossible to ensure type safety without memory safety

Buffer overflow in Java or C#

Ruled-out at the language level by combination of

- compile-time (static) type checking
  - also at *load-time* by *bytecode verifier* (bcv)
- runtime (dynamic) checks

what runtime checks are performed during the execution of

```
public class A extends Super{
  protected int[] d;
  private A next;
  public A() { d = new int[3] ; }
  public void m(int j) { d[0] = j ;}
  public setNext(Object s) {
       next = (A)s ;
  }
}
```

runtime check for non-nullness of d and array bound

runtime check for type downcast

Buffer overflow in Java or C#

Buffer overflow can still be present

- in native code
- in C# in code blocks declared unsafe
- through bugs in the Virtual Machine (VM) implementation (typically written in C++)
- through bugs in the implementation of the type-checker or in the type system (*unsound*)

The *VM* (along with the *bcv*) is part of the *Trusted Computing Base* (TCB) for both *memory and type safety*

How to break type safety?

Type safety is not a robust property

Data values and objects are just memory locations.

If type confusion is created (e.g., by *having different references with different types referring to the same memory locations*), there can be NO type guarantees

Attack on Java in Netscape 3.0

```
public class A[] { ... }
```

Netscape's Java execution engine confused this type with the type

```
array of A
```

Input validation problem: use of [and] in class names

Another type confuson attack

```
public class A {
...
  public Object x;
...
}
```

```
public class A {
  public int x;
...
}

public class B {
  void setX(A a) {
     a.x = 12 ;
  }
}
```

What happens if B is compiled against A but run against A? *Pointer arithmetic*!

If JVM allowed binary incompatible classes to be loaded, the type system would break

How to know a type system is sound?

Representation independence (for booleans)

- no difference if `true` is represented as `0` and `false` as `1` (or `FF`) or viceversa
  - given program with either representation guaranteed to produce same result
- test for it or prove it
  - given formal definition of language, prove that the representation of `true/false` has no effect on any program
- similar properties should hold for all datatypes

How to know a type system is sound?

Give two formal definitions of the programming language

- a *typed* operational semantics, which records and checks type information at runtime
- an *untyped* operational sematics which does not

and prove their equivalence for all *well-typed* programs

in other words, prove the equivalence of

- a defensive execution engine (which records and checks all type information at runtime) and
- a normal execution engine which does not for any program

accepted by the type checker

Ongoing evolution to richer type systems

Many ways to enrich type systems (further)

- distinguish non-null and possibly-null types

```
public @nonNull String hello = "hello";
```

to

  - improve efficiency
  - prevent null pointer bugs or catch them earlier, at compile time
- alias control to restrict interferences due to 'double names'
- information flow to control the way tainted information flows through

Other language-based guarantees

- visibility : `public, private`, etc
- immutability
  - of primitive values (constants)
    - in Java: **`final int i = 4;`**
    - in C(++): **const int `BUFF_SIZE=128;`**
      - meaning of const confusing for C(++) pointers and objects!
  - of objects
    - in Java, **String** objects are constant

Safe arithmetic

What happens if `i=i+1;` overflows ?

1. *Unsafest approach*: leaving this as an undefined behavior (as in C and C++)
2. *Safer approach*: specify how over/under flow behaves (as in Java and C#)
3. *Safer still:* integer overflow results in an exception (as in checked mode in C#)
4. *Safest approach:* have infinite precision integers and reals, so that overflow never happens (in some experimental functional languages)

(Lack of) thread safety

Two concurrent execution threads executing the same statement

```
x = x+1;
```

at the end, x can have value 2 or 1

Cause: data race since `x = x+1;` is **not** an atomic operation, but consists of two steps which may be interleaved in unexpected ways

(and can cause security problems ...)

Multi-threading behavior in Java

```
class A {
   private int i;
   A() {i=4;}
   int geti() {return i;}
}
```

Can `geti()` ever return something different that 4 ?

Thread 1 initializing `x`

```
static A x = new A();
```

Thread 2 accessing `x`

```
j = x.geti();
```

Multi-threading behavior in Java

```
class A {
   private int i;
   A() {i=4;}
   int geti() {return i;}
}
```

Can `geti()` ever return something different that 4 ?

Thread 1 initializing `x`

```
static A x = new A();
```

Thread 2 accessing `x`

```
j = x.geti();
```

Execution of thread 1 takes 3 steps

1. allocate `new object m`
2. `m.i = 4;`
3. `x = m;`

compiler or VM can swap 2. and 3. since independent
Hence in thread 2, `x.geti();` can return 0 instead of 4

(Lack of) thread safety

```
class A {
   private final int i;
   A() {i=4;}
   int geti() {return i;}
}
```

Now `geti()` always returns 4

Declarng a private field as **`final`** fixes this particular problem (because of ad-hoc restrictions on the initialization of final fields)

only since the revision (2004) of Java Memory Model, which specifies how compilers and VM can deal with concurrency

API implementation of **`String`** was fixed only in Java 2 (aka 1.5)

Data races and thread safety

- a program contains a data race if two threads simultaneously access (not in read only) the same variable
- **thread-safety** = the behavior of a program consisting of several threads can be understood as interleaving of those thread
- in Java, the semantics of programs with data races is effectively undefined, so only programs without data races are thread-safe

- MORAL: even “safe” programming languages can sport weird behavior in presence of concurrency

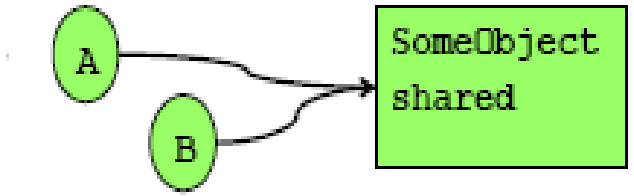
Typing breaks in C, Java, C# …

Dangerous combinations: aliasing & mutation

threads or objects A and B

both have references to a

mutable object `shared`



this can cause many problems, not just with concurrency

1. in concurrent (multi-threaded) context: data races
   - locking objects (as with `synchronized` in Java) can help but is expensive and risks deadlock
2. in single-threaded context: dangling pointers
   - who is responsible for free-ing `shared`? A or B ?
3. in single-threaded context: broken assumptions
   - if A changes the `shared` object, B's assumptions about shared may not hold and B's code broken

Danger of references to mutable data

In multi-threaded programs, references to mutable data structures can be a problem, since referenced data can change even in safe languages (like Java or C#)

```
public void f(char[] x) {
    if (x[0] != 'a' {throw new Exception();}
    // cannot assume first element is 'a'
    ...
}
```

Another thread with reference to `x` can change the content of the array at any moment, also just after the if-statement has been executed

References to immutable data less dangerous

In multi-threaded programs, references to immutable data structures are safer

```
public void g(String x) {
    if (x.charAt(0) != 'a' {throw new Exception();}
    // can assume here that first character is 'a'
    ...
}
```

Another thread with a reference to the same string x cannot change the content of the array (value of the string) since Java strings are immutable

"Secure" programming languages

- Languages like Java and .NET can provide security guarantees in the presence of untrusted code
- the languages can guarantee restrictions on the possible interactions between different modules by using a combination of
  - typing
  - visibility
- enforced by a combination of static and runtime checks
  - static checks at load-time rather than at compile-time